



Veille CVE

Agent de monitoring automatisé

Python ▪ Docker Compose ▪ Trivy ▪ Slack

Auteur

HONORINE Kylian

Module

Stage — Infra & Cybersécurité 2025/2026

Sommaire

1	Contexte	1
2	Sources de veille	2
3	Architecture & boucle de veille	2
4	Filtrage & priorisation	3
4.1	Matching sur la stack surveillée	3
4.2	Grille de priorité P0–P4	3
5	Déduplication & anti-flood	3
6	Sécurité de l’agent & observabilité	3
7	Résultats concrets	4
8	Synthèse technique	4
9	Limites & évolutions	4
10	Conclusion	4

1 Contexte

 Mission

Automatiser la phase « **identifier** » d’une procédure de maintenance applicative sur l’infrastructure Kubernetes RKE2, inspirée du **NIST SP 800-40** (gestion des correctifs) et du guide **ReCyF de l’ANSSI** : construire un agent qui détecte, priorise et notifie les vulnérabilités touchant la stack en production.

Sans veille continue et fiable, les trois phases suivantes de la procédure — évaluer, corriger, vérifier — restent aveugles. Ce projet a été mené **en autonomie complète pendant un stage**, sans accompagnement technique de l’entreprise sur ce sujet précis : identification des sources, conception de l’architecture, logique métier, audit de sécurité de l’outil lui-même.

Stack technique :

- **Python** (~1100 lignes) — logique de récupération, matching et notification
- **Docker Compose** — conteneurisation et déploiement en production
- **Trivy** — scan de vulnérabilités des images Docker
- **Prometheus / Grafana** — observabilité de l’agent
- **Claude (IA générative)** — assistant de développement, sous ma supervision

2 Sources de veille

Cinq sources officielles, interrogées via leurs **API** plutôt que par scraping — plus rapide, plus stable, respectueux des CGU et indépendant du design des pages.

Source	Rôle	Fréquence
CISA KEV	CVE confirmées activement exploitées	15 min
NVD (NIST)	Base mondiale des CVE, score CVSS	30 min
CERT-FR (ANSSI)	Avis de sécurité France / Europe	30 min
GHSA (GitHub)	Vulnérabilités des dépendances open source	1 h
endoflife.date	Fins de support produits	1 / jour
Trivy (images Docker)	Scan CVE de 9 images de la stack monitoring	1 / jour

Pourquoi des fréquences différenciées : KEV est la source la plus actionable (exploitation confirmée) donc pollée le plus souvent ; Trivy est volontairement limité à une fois par jour pour ne pas saturer le canal Slack.

3 Architecture & boucle de veille

L'agent tourne en **mode daemon continu** dans un conteneur Docker, chaque source étant pollée selon sa propre fréquence.

```
SOURCES = {
    "cisa_kev": {"interval": 15 * 60},
    "nvd": {"interval": 30 * 60},
    "cert_fr": {"interval": 30 * 60},
    "ghsa": {"interval": 60 * 60},
    "endoflife": {"interval": 24 * 3600},
    "trivy_images": {"interval": 24 * 3600},
}

def run_forever():
    while True:
        for name, cfg in SOURCES.items():
            if due(name, cfg["interval"]):
                items = fetch(name) # appel API dedie
                candidates = match_stack(items) # filtrage TEE0
                for cve in candidates:
                    priority = score_priority(cve)
                    if should_notify(cve, priority):
                        notify_slack(cve, priority)
                sleep(POLL_TICK)
```

Rôle de l'IA générative

Le code Python a été généré avec l'assistance de **Claude**. Mon rôle : architecte et testeur — spécifications fonctionnelles, logique métier, validation de chaque itération, correction des bugs découverts en production. L'IA a accéléré l'écriture, la maîtrise du fonctionnement reste entièrement mienne.

4 Filtrage & priorisation

4.1 Matching sur la stack surveillée

Environ 30 000 CVE sont publiées chaque année — l'immense majorité sans rapport avec l'infrastructure réelle. Le matching combine deux mécanismes :

- **Matching CPE** (Common Platform Enumeration) — identifiants normalisés NVD, fiable et sans faux positif ;
- **Matching textuel** — recherche des ~179 composants de la stack (Kong, Keycloak, Vault, PostgreSQL, Grafana, Kubernetes...) dans titre/description, utilisé en repli quand le CPE est absent.

4.2 Grille de priorité P0–P4

Inspirée de la directive **CISA BOD 22-01**, la priorité combine score CVSS, statut KEV et exposition du composant (public via Kong/Keycloak, ou interne au cluster).

Prio.	Critère dominant	SLA
P0	CVSS élevé + KEV + composant exposé	24–72 h
P1	CVSS élevé ou KEV, exposition partielle	14 jours
P2	CVSS moyen, composant interne critique	30 jours
P3	CVSS moyen, faible exposition	60 jours
P4	CVSS faible / composant peu critique	90 jours

5 Déduplication & anti-flood

Une même CVE apparaît souvent dans plusieurs sources à quelques jours d'écart. Sans garde-fou, l'équipe recevrait plusieurs alertes redondantes pour la même faille — fatigue d'alerte garantie.

- **Base SQLite** des CVE déjà notifiées — une CVE revue dans une autre source est ignorée par défaut ;
- **Re-notification sur upgrade** — si le statut passe en KEV ou si la priorité augmente, une nouvelle alerte part avec un bandeau « PRIORITÉ ÉLEVÉE » ; l'inverse (priorité qui baisse) ne re-notifie jamais ;
- **Mode baseline** au premier démarrage — ingestion silencieuse des 500 dernières CVE, sans notification, pour éviter un flood initial de 50+ messages (pattern standard : warmup Prometheus, learning mode SIEM/EDR).

6 Sécurité de l'agent & observabilité

Un outil de sécurité doit être sécurisé lui-même

Protection SSRF sur les liens suivis, exécution en utilisateur **non-root**, validation stricte des noms d'image Docker (anti-injection via Trivy), fail-fast au démarrage si une variable d'environnement critique manque.

L'agent expose **14 métriques Prometheus** couvrant les quatre Golden Signals SRE (latence, trafic, erreurs, saturation), visualisées dans un dashboard Grafana à 7 sections. Une CLI (**docker**

`exec`) permet diagnostic et tests : `--stats`, `--history`, `--test-pipeline`.

7 Résultats concrets

Après deux mois de production continue (24 h/24) :

- Remontée d'une CVE **Palo Alto** en priorité P0 via le flux KEV ;
- Détection de la faille **ArgoCD ServerSideDiff** (CVSS 9.6), ayant déclenché une planification de mise à jour ;
- Détection d'images Docker cumulant plus de 100 CVE HIGH, motivant un changement de stratégie d'image de base ;
- Auto-surveillance : alerte Slack si l'agent lui-même reste silencieux plus de 4 h.

8 Synthèse technique

Bloc	Rôle
Fetchers API	Interrogation des 5 sources CVE à fréquence dédiée
Trivy runner	Scan quotidien de 9 images Docker via subprocess
Matching engine	Filtrage CPE + textuel sur la stack surveillée
Priority engine	Score P0–P4 (CVSS + KEV + exposition)
Dedup store	SQLite, re-notification sur upgrade uniquement
Notifier	Webhook Slack + backup SMTP
Observability	14 métriques Prometheus, dashboard Grafana, CLI

9 Limites & évolutions

Liste des 9 images Trivy maintenue manuellement — l'évolution naturelle serait un **Trivy Operator** déployé dans le cluster pour une couverture exhaustive automatique. Absence de ticketing (Jira/Asana) pour le suivi du cycle de vie des vulnérabilités, attendu par **NIS2**. Pas de détection runtime (**Falco**) ni d'audit de configuration Kubernetes (**Kubescape**) — deux angles morts identifiés pour une industrialisation future.

10 Conclusion

Ce projet démontre qu'un outil de veille cybersécurité robuste peut être conçu et livré en autonomie, en s'appuyant sur l'IA générative comme accélérateur de développement sans jamais en déléguer la logique métier ni la responsabilité. Le résultat : un agent qui transforme un flux brut de ~30 000 CVE annuelles en une poignée d'alertes réellement actionables.

« Sans veille continue, la correction arrive toujours trop tard. »